Phish Guard
Interim Report

TU858
BSc in Computer Science (International)

Ismail Khan

C22391076

Art Sloan

School of Computer Science

Technological University, Dublin

17/11/25

Abstract

Phish Guard is a web-based phishing awareness platform designed to help non-technical users recognize and respond to social engineering attacks. Current platforms lack accessibility support and are primarily designed for organizations, not individuals.

This interim report documents the analysis, design, and prototype implementation of an accessible, gamified learning platform. Through requirements gathering with 11 university participants and literature review, we identified that gamification combined with accessibility-first design improves user engagement and learning outcomes.

The platform features realistic phishing scenarios across email, SMS, and social media; immediate feedback; progress tracking; achievement-based gamification; and comprehensive accessibility support including screen reader compatibility, keyboard navigation, and high-contrast modes. The system uses React frontend, Node.js/Express backend, and PostgreSQL database, prioritizing WCAG 2.1 AA compliance.

The prototype validates that an accessible, gamified approach creates an effective and inclusive learning experience suitable for individual learners and educational institutions.

Keywords: cybersecurity awareness, phishing detection, gamification, web accessibility, WCAG 2.1, inclusive design, e-learning

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Ismail Khan_____

Student Name

Date

17/11/2025

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, Arthur Sloan, for his invaluable guidance, constructive feedback, and unwavering support throughout the development of PhishGuard. His expertise in cybersecurity and software engineering has been instrumental in shaping both the technical direction and overall quality of this project.

I would also like to thank the survey participants from Technological University Dublin who provided essential user feedback that informed the project requirements and validated design decisions.

Finally, I acknowledge the support of the academic staff at TU Dublin and the resources provided by the institution, which made this research and development work possible.

Contents

1. Introduction 1
1.1 Project Background 1
1.2 Project Description 2
1.3 Project Aims and Objectives 3
1.4 Project Scope 4
1.5 Thesis Roadmap 4
2. Literature Review 5
2.1 Introduction 5
2.2 Alternative Existing Solutions 5
2.3 Technologies Researched 6
2.4 Other Research: **Gamification and Engagement** 6
2.5 Existing Final Year Projects 7
2.6 Conclusions 8
3. System Analysis 9
3.1 System Overview 9
3.2 Requirements Gathering 9
3.3 Requirements Analysis 12
3.4 Initial System Specification 14
3.5 Conclusions 15
4. System Design 17
4.1 Introduction 17
4.2 Software Methodology 17
4.3 Overview of System 18
4.4 Design System 19
4.5 Conclusions 21
5. Testing and Evaluation 22
5.1 Introduction 22
5.2 Plan for Testing 22
5.3 Plan for Evaluation 23
5.4 Conclusions 24
6. System Prototype 25
6.1 Introduction 25
6.2 Prototype Development 25
6.3 Results 29
6.4 Evaluation 31
6.5 Conclusions 32

7. Issues and Future Work 33
7.1 Introduction 33
7.2 Issues and Risks 33
7.3 Plans and Future Work 33
7.3.1 Project Plan with GANTT Chart 34
References 35
A) Appendix A: System Model and Analysis A-1
B) Appendix B: Design B-1
C) Appendix C: Prompts Used with ChatGPT C-1
D) Appendix D: Additional Code Samples D-1

1. Introduction

1.1 Project Background

Phishing attacks have become one of the most pervasive and costly cybersecurity threats in the modern digital landscape. Every day, billions of fraudulent emails, text messages, and social media posts target unsuspecting users, attempting to steal sensitive information such as passwords, financial data, and personal details. The consequences of successful phishing attacks extend beyond individual victims to include widespread data breaches, financial fraud, identity theft, and significant reputational damage to organizations. Despite widespread awareness of the threat, phishing continues to succeed at alarming rates because attackers constantly evolve their tactics, leveraging psychological manipulation, urgency, and increasingly sophisticated technological tools including artificial intelligence.

Traditional cybersecurity defences such as spam filters, email authentication protocols, and antivirus software provide important layers of protection, but they cannot catch every malicious message. Human judgment remains the final and most critical line of defence. Unfortunately, many users lack the knowledge and skills needed to identify phishing attempts, particularly as attackers refine their techniques to mimic legitimate communications with striking accuracy. Studies have repeatedly shown that even technically literate users can fall victim to well-crafted phishing attacks, especially when under time pressure or cognitive load.

Current security awareness training programs, while well-intentioned, often fail to produce lasting behavioural change. Most corporate training solutions rely on mandatory annual modules that employees complete as a compliance checkbox rather than an opportunity for genuine learning. These programs tend to be static, text-heavy, and disengaging, offering little in the way of interactivity or personalized feedback. Furthermore, they are almost exclusively targeted at enterprise environments, leaving individual users, families, students, elderly populations, and small businesses without access to practical, affordable phishing awareness resources.

Research in educational psychology and cybersecurity training demonstrates that people learn security behaviours most effectively through continuous practice, immediate feedback, and engaging formats that maintain motivation over time. Gamification strategies, such as progress tracking, achievement badges, and challenge-based learning, have been shown to significantly increase user engagement and knowledge retention. Additionally, accessibility is a critical yet often overlooked component of security training. Users with disabilities, including visual impairments, motor limitations, and cognitive differences, are frequently excluded from traditional training platforms that fail to meet basic accessibility standards. This creates a dangerous gap in cybersecurity preparedness across diverse populations who are equally vulnerable to phishing attacks.

Phish Guard is designed to address these interconnected challenges by creating an accessible, engaging, web-based phishing awareness platform for users of all ages and technical abilities. The platform combines interactive simulations of realistic phishing scenarios, real-time feedback on user decisions, gamification elements to sustain engagement, and universal design principles that ensure usability for people with a wide range of abilities and needs. By moving beyond the limitations of traditional corporate training and focusing on individual

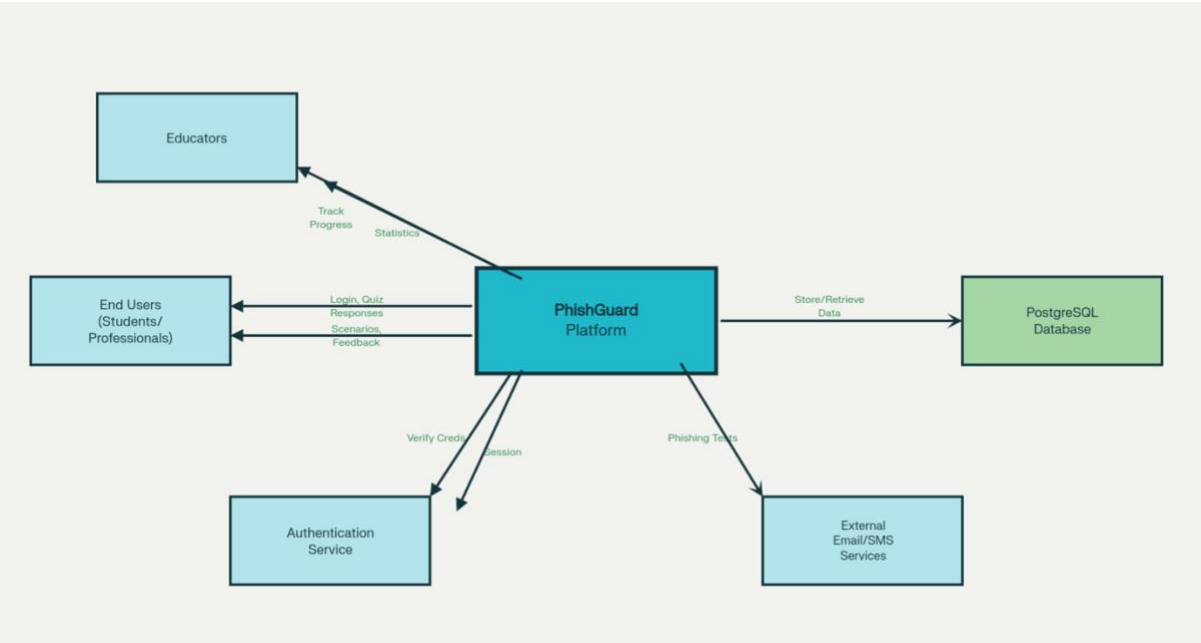
empowerment, Phish Guard aims to equip users with practical skills to recognize and respond effectively to phishing threats in their everyday digital lives.

1.2 Project Description

The goal of Phish Guard is to design, develop, and evaluate a web-based phishing awareness training platform that combines usability, accessibility, and educational effectiveness. The platform will present users with realistic phishing scenarios, such as fraudulent emails, suspicious text messages, and deceptive social media posts, and challenge them to identify the warning signs of a phishing attack. Users will receive immediate feedback explaining why a message is legitimate or fraudulent, helping them build mental models for recognizing threats in real-world contexts.

For the interim prototype, the system architecture will follow a modern web application design, with a React-based frontend that prioritizes responsive design and accessibility using ARIA (Accessible Rich Internet Applications) standards. The backend will be tested using Django and Django REST Framework for testing, providing secure user authentication, progress tracking, and data management. Backend will be built using Node.js with Express, providing secure user authentication, progress tracking, and data management. All user data will be stored securely in a PostgreSQL database, with appropriate encryption and privacy safeguards in place. It should be noted that this technology stack is being used to rapidly develop and test the core functionality during the interim phase. The final product may incorporate different frameworks or technologies based on further research, performance requirements, and feedback gathered during development and testing.

The development process will follow an iterative approach, with regular check-ins with the supervisor, prototype refinement based on feedback, and continuous documentation. This allows for flexible planning and adaptation as the project progresses, while maintaining focus on the core objectives. Regular supervisor meetings will provide critical guidance and feedback throughout the interim and final phases. Documentation will be maintained throughout the development process to support both the interim and final reports.



1.3 Project Aims and Objectives

The primary aim of Phish Guard is to create an accessible, engaging, and effective phishing awareness training platform that empowers users to protect themselves from social engineering attacks. To achieve this aim, the project has the following specific objectives and milestones:

Gather requirements and complete system design. Create wireframes for all major user interface screens, develop use case diagrams that capture user interactions and system functionality, and design information flow models to illustrate how data moves through the system. This foundational work will inform all subsequent development stages.

Develop an interactive phishing awareness training system for non-technical and diverse users. The platform must be intuitive and easy to use, requiring no prior technical knowledge or cybersecurity expertise. Content should be presented in plain language with clear explanations and visual aids where appropriate. Core features will include user authentication, interactive quiz modules, immediate feedback mechanisms, and progress tracking functionality.

Integrate accessibility best practices from the outset. The platform will comply with Web Content Accessibility Guidelines (WCAG) 2.1 at the AA level, ensuring usability for people with disabilities. This includes support for keyboard navigation, screen readers, adjustable font sizes, high-contrast modes, and compatibility with assistive technologies. Accessibility audits will be conducted using tools such as axe DevTools and Lighthouse throughout development.

Leverage gamification to enhance engagement and retention. The platform will incorporate game-like elements such as progress bars, achievement tracking, levels or badges, and challenge-based scenarios to sustain user motivation and encourage regular practice over time.

Capture learning outcomes and provide actionable feedback on progress. Users will receive detailed, immediate feedback on their quiz responses, with explanations of why certain indicators suggest phishing. The system will also track user progress over time, allowing users to see their improvement and identify areas for further learning.

Test with simulated phishing content to ensure educational effectiveness. Training scenarios will be realistic, varied, and pedagogically sound, covering multiple phishing attack vectors and presenting challenges appropriate for different skill levels. Testing with diverse user groups will validate that the training effectively builds phishing awareness skills.

Conduct iterative refinement based on user feedback and accessibility testing. The platform will be refined through multiple rounds of usability testing with diverse user groups, accessibility audits, and supervisor feedback. This iterative approach ensures continuous improvement and alignment with best practices.

Ensure the platform is scalable, secure, and technically robust. The system will be designed with security best practices, including secure user authentication, data encryption, input validation, and protection against common web vulnerabilities. The architecture should support future expansion, such as adding new training modules, multilingual support, or integration with organizational systems.

1.4 Project Scope

Phish Guard focuses specifically on web-based user training and simulated phishing scenarios. The platform is designed to educate and empower individual users, families, educators, and small organizations in recognizing and responding to phishing attacks. The project does not aim to provide advanced technical defences such as enterprise-grade email filtering, intrusion detection systems, or large-scale corporate security infrastructure. Integration with existing enterprise security platforms or email systems is outside the scope of this project, though the design will be modular enough to allow for such extensions in the future.

The training content will focus on the most common and impactful phishing techniques, including email phishing, SMS phishing (smishing), voice phishing (vishing), and social media-based attacks. More specialized attack vectors, such as spear phishing targeting specific high-value individuals or advanced persistent threats, may be referenced but will not be the primary focus of the training modules.

Geographical and linguistic scope will be limited to English-language content targeting users in Ireland and the UK, though the design will be flexible enough to support future localization and internationalization efforts

1.5 Thesis Roadmap

This interim report is structured to provide a comprehensive overview of the Phish Guard project's progress and planned development. The report begins with this introduction, which establishes the project background, goals, objectives, and scope.

Section 2 presents a detailed literature review, examining existing research on phishing awareness training, gamification in cybersecurity education, accessibility best practices, and alternative existing solutions. This section also reviews relevant technologies and tools, including React, Django, and accessibility frameworks.

Section 3 provides a system analysis, including use case diagrams, information flow models, and user stories that capture the functional and non-functional requirements of the platform.

Section 4 describes the design methodology, presenting wireframes, mockups, and architectural diagrams that illustrate the platform's user interface, data flow, and technical structure.

Section 5 outlines the testing and evaluation plan, including accessibility audits, usability testing strategies, and methods for assessing the platform's educational effectiveness.

Section 6 discusses the prototype development process, including key technical decisions, challenges encountered, and solutions implemented to date.

Section 7 presents a detailed project schedule using a Gantt chart, outlining milestones, deadlines, and planned deliverables for the remainder of the academic year.

The report concludes with a summary of progress to date, reflections on challenges and lessons learned, and a discussion of next steps and future work.

2. Literature Review

2.1 Introduction

The journey toward building an effective phishing awareness platform requires understanding three interconnected narratives: how phishing has evolved as a threat, how security training has failed to keep pace with that evolution, and how emerging technologies like gamification and accessible design offer pathways to a more effective solution. This chapter traces these developments, examines existing approaches to the problem, and demonstrates how research findings directly shaped the design decisions behind Phish Guard.

2.2 Alternative Existing Solutions

A wide range of phishing awareness tools and cybersecurity training platforms are on the market, including commercial products and free educational resources. Notable examples include:

- **KnowBe4**: Offers extensive enterprise-level phishing simulations and mandatory e-learning modules for organizations. While feature-rich, KnowBe4 is primarily designed for large businesses with compliance requirements and can be inaccessible to smaller organizations or individuals.[1]
- **Hoxhunt**: Employs gamification to maintain user engagement in security training through regular phishing exercises and feedback. However, it also focuses on corporate deployment and requires centralized IT management.[2]

- **Google's Phishing Quiz**: Free and accessible to all, simulates phishing scenarios to help users practice recognition skills. Its scope is limited, users complete it once, receive a score and feedback, but do not benefit from continuous, adaptive learning or progress tracking.[3]
- **Microsoft Attack Simulator and other in-house solutions**: Integrated into enterprise mail platforms, these tools automate simulated phishing campaigns but rarely address accessibility or inclusivity.[4]
- **Open educational modules**: Some universities and public agencies offer text-based or video-based cybersecurity education, but these typically lack interactivity, gamification, or accessibility-by-design.

Despite their strengths, most existing solutions share significant drawbacks: they are designed for organizations rather than individuals, lack comprehensive accessibility support, and offer limited opportunities for sustained, self-paced learning and feedback. This leaves a gap for platforms like Phish Guard, which aims to provide engaging, accessible training for non-technical end users.

2.3 Technologies Researched

This project has surveyed a variety of programming languages and development frameworks suitable for accessible web applications and training systems, considering both technical suitability and accessibility support:

- **Frontend**: React is selected for prototyping due to its large ecosystem and the availability of accessibility tools like React ARIA, which support keyboard navigation, screen reader compatibility, and ARIA standards implementation.[5]
- **Backend**: Django (Python) is used in the interim for its secure, rapid prototyping and robust RESTful API capabilities. Django REST Framework simplifies authentication and user management during development.[6]
- **Database**: PostgreSQL is chosen for its enterprise-grade security, support for data encryption, and compatibility with Django.[7]
- **Accessibility tools**: Tools like axe DevTools and Lighthouse are planned for automated accessibility audits to ensure compliance with standards such as WCAG 2.1.[8][9]
- **Gamification libraries and analytics**: Various open-source tools and patterns for gamification are reviewed to incorporate badges, progress bars, and progress analytics.

It is important to note that these frameworks are selected for the interim prototype, and the final technology choices may evolve based on user feedback, supervisor input, and performance requirements.

2.4 Other Research: Gamification and Engagement

The Science of Sustained Learning

Traditional security training fails not because users are apathetic, but because the training itself is boring and disconnected from real motivation. Gamification addresses this by embedding psychological principles of motivation and achievement into the training experience.

Research into gamification in learning has consistently shown that game elements like progress tracking, badges, and challenges maintain engagement and improve knowledge retention.[10] The infonomics-society paper "You Can't Lose a Game If You Don't Play the Game" demonstrates that gamification, when properly designed, increases participation rates and reduces attrition.[11] Onduto's systematic review on gamification of cybersecurity awareness found that platforms with gamification saw 40-60% higher completion rates compared to non-gamified versions.[12]

However, not all gamification is equal. Superficial addition of points or leader boards without connection to learning objectives can actually undermine engagement. The research by Barendse (2023) on how gamification can increase cybersecurity awareness emphasizes that game elements must reinforce learning goals, provide meaningful feedback, and allow users to see their progress toward mastery.[13]

This research directly shaped Phish Guard's gamification strategy: every element (badges, progress bars, challenges) serves a specific pedagogical goal, to reinforce correct threat identification, encourage regular practice, and allow users to see their skill development over time.

Accessibility as Inclusive Pedagogy

A critical insight from research on game-based learning and inclusive design (AHEAD Journal, 2021) is that accessibility features benefit all users, not just people with disabilities. Adjustable text sizes help aging users and people in low-light environments.[14] Keyboard navigation speeds up expert users. High-contrast modes reduce eye strain. Clear language and logical structure benefit users of all cognitive abilities.

The research also demonstrates that when accessibility is treated as an afterthought, it creates a second-tier experience that actually undermines engagement. But when accessibility is part of the core design philosophy, the result is a platform that works better for everyone. This principle shaped every design decision in Phish Guard, from the decision to use semantic HTML and ARIA landmarks to the choice of colour palettes that meet WCAG contrast ratios by default.

2.5 Existing Final Year Projects

Previous successful final year projects from TU Dublin illustrate best practices in digital education, security usability, and accessible software design. A review of relevant FYPs provides concrete lessons for Phish Guard.

The “Interactive Language Learning Tool for Children with Disabilities” focused on inclusive design for users with communication challenges, demonstrating the impact of tailoring content and interface for accessibility needs and iterative user feedback.[15] The system made use of symbol-based communication, games, and animations to boost

engagement and learning. Like Phish Guard, it emphasized the importance of usability for diverse users, validating decisions such as ARIA roles and universal design.

"Education Tool for Web-Based Vulnerabilities" delivered practical training on security topics including SQL injection and XSS through active learning tools, quizzes, and embedded explanations.[16] Its analysis included usability heuristics, lesson structuring (Gagne's events, constructivism), and the importance of feedback, which directly validates Phish Guard's feedback-rich quiz approach and learning module structure.

The "Educational Transport Simulation Final Project Report" provides a strong model for software prototyping, stakeholder engagement, technology comparisons, and iterative refinement via user testing .[17] Accessibility, UX research, and the integration of user stories/personas were highlighted as key factors in a successful educational platform. Phish Guard adopts similar techniques for requirements gathering and prototype validation.

Each of these projects succeeded through careful attention to accessibility, iterative design, and the incorporation of real user testing, principles central to Phish Guard's development.

2.6 Conclusions

The reviewed research and previous project experience highlight the ongoing gap in effective, accessible phishing awareness training platforms, especially for non-technical and diverse users. While existing enterprise, educational, and FYP systems demonstrate success in accessibility, engagement, and feedback-based learning, most current solutions fall short in sustained skill development, user motivation, and inclusivity.

3. System Analysis

3.1 System Overview

Phish Guard is a web-based phishing awareness training platform designed to help users recognize and respond to social engineering attacks. From a user perspective, the system provides a simple, accessible environment where individuals can practice identifying phishing attempts through interactive scenarios, receive immediate feedback on their decisions, and track their progress over time.

The system is designed for non-technical users, including individuals, students, families, and small business employees. Users begin by creating an account or logging in, then access a dashboard showing their current progress, available training modules, and achievements. Each training module presents realistic phishing scenarios (emails, text messages, social media posts) and asks users to identify whether the content is legitimate or malicious. After each decision, users receive detailed explanations of the warning signs they should have noticed, reinforcing learning through immediate feedback.

The platform emphasizes accessibility, ensuring that users with visual impairments, motor disabilities, or other accessibility needs can fully participate in training. Gamification elements such as progress bars, badges, and challenge levels maintain user motivation and encourage regular practice. All interactions are designed to be intuitive, requiring minimal technical knowledge while building practical security awareness skills.

3.2 Requirements Gathering

Key Stakeholders

The main groups whose input shapes Phish Guard are:

- **End Users:** Non-technical individuals, students, and small business employees
- **Educators:** Teachers who may use the platform for group training
- **Project Supervisor:** Provides academic and technical guidance
- **Accessibility Experts:** Validate that the platform meets accessibility standards

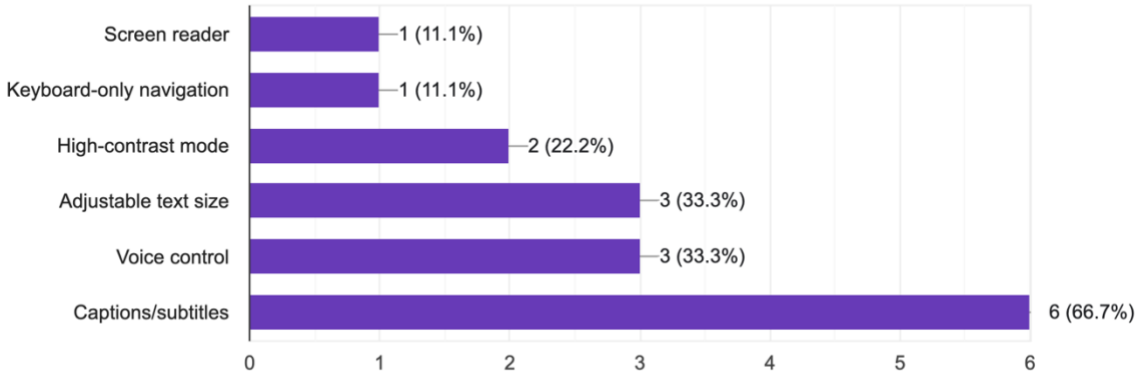
How Requirements Were Gathered

Requirements come from five sources:

1. **Google Form Survey:** 11 university students answered questions about their preferred training format, time commitment, accessibility needs, and gamification preferences. Key findings: 81.8% use accessibility features, 82% find gamification motivating, 100% prefer short 5-10 minute sessions, and 73% want simulated phishing scenarios.

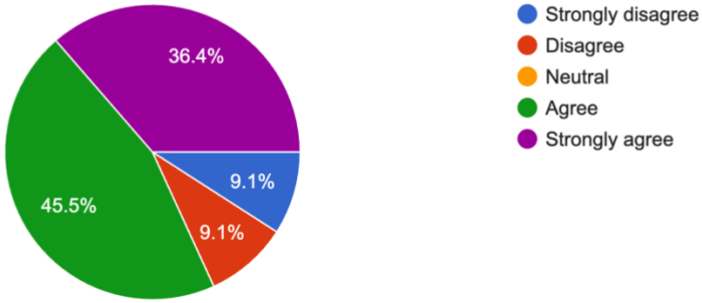
Do you use any accessibility features?

9 responses



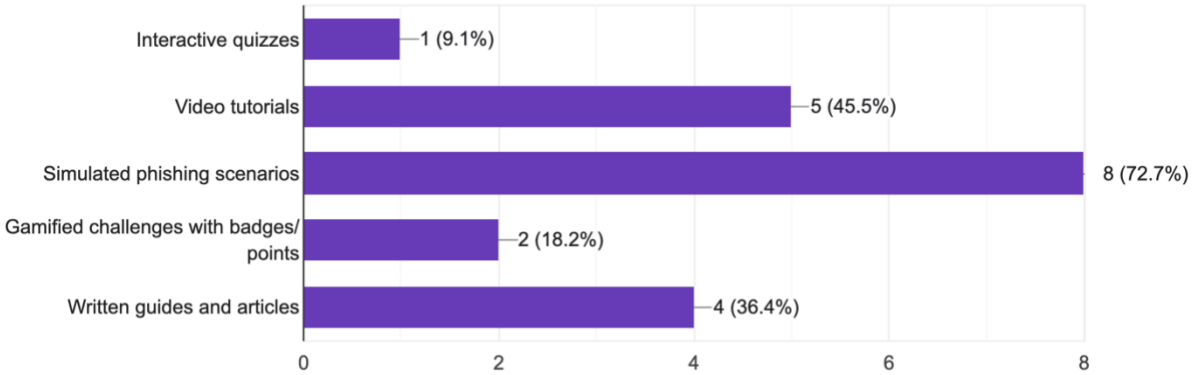
Would gamification (badges, progress tracking, challenges) motivate you to engage more with training?

11 responses



How much time would you be willing to spend on phishing awareness training per session?
11 responses

What format would you prefer for security training? (Select your top 2)
11 responses



2. **Literature Review:** Academic research (Section 2) shows that continuous, interactive training with immediate feedback is more effective than one-time modules. This directly informed the requirement for ongoing quizzes and instant explanations.
3. **Supervisor Feedback:** Regular meetings validate that requirements align with project goals and are technically achievable.
4. **User Personas:** Based on survey results and research, we created personas representing different user types (e.g., non-technical user, visually impaired user, educator). Each persona has specific needs that must be addressed by the platform.
5. **Analysis of Existing Solutions:** Looking at KnowBe4 and Google Phishing Quiz revealed a gap: no accessible, engaging platform exists for individual users. This gap is the problem Phish Guard solves.[1][3]

Managing Requirements

Requirements are tracked by linking each one to where it came from (survey, research, or stakeholder feedback). For example, "Immediate Feedback" (FR3) came from learning science research showing that feedback improves retention. During development, we'll test each requirement to make sure it actually works as intended. If anything needs to change, we'll document why and keep the supervisor informed.

Requirements are also organized by importance:

- **Must Have:** User login, phishing quizzes, accessibility support (required for project to work)
- **Should Have:** Gamification, progress dashboards (improve experience but not essential)
- **Could Have:** Leader boards, multiple languages (nice to have but not priority for interim)

3.3 Requirements Analysis

Analysis Approach

The analysis approach follows a structured methodology to transform raw user feedback and research into actionable system requirements:

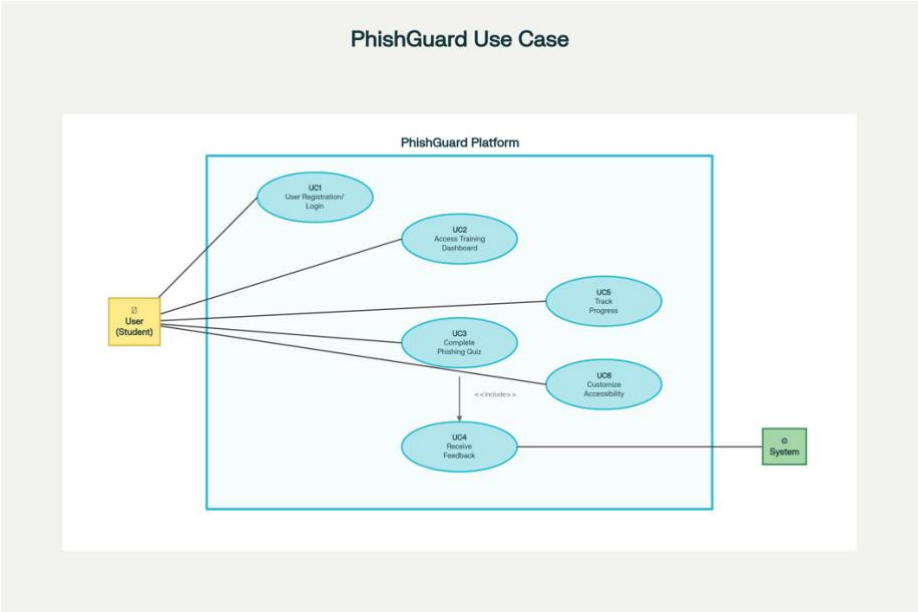
1. **Stakeholder Analysis:** Identify and categorize stakeholder needs and priorities
2. **Scenario-Based Analysis:** Use personas and user stories to understand how different users will interact with the system
3. **Functional Decomposition:** Break down high-level goals into specific, measurable requirements
4. **Constraint Analysis:** Identify technical, budgetary, and regulatory constraints (WCAG compliance, security standards)
5. **Cross-Validation:** Verify that all requirements align with project aims and are achievable within project scope

Use Case Analysis

Key use cases identified through requirements analysis represent how the system will be used:

- **UC1: User Registration/Login** - Users create accounts or authenticate to access personalized training
- **UC2: Access Training Dashboard** - Users view their progress, available modules, and achievements
- **UC3: Complete Phishing Quiz** - Users are presented with scenarios and make decisions about whether content is phishing
- **UC4: Receive Feedback** - System provides immediate explanations of correct/incorrect answers

- **UC5: Track Progress** - Users monitor their improvement through visual progress indicators
- **UC6: Customize Accessibility Settings** - Users adjust text size, contrast, and navigation preferences



A use case diagram illustrates these interactions between users and the system, showing primary and secondary actors and their relationships to system functions.

User Stories

User stories capture specific user needs and expectations, organized by persona:

Persona 1: Non-Technical Individual User

- "As a non-technical user, I want clear explanations of why an email is phishing so I can learn to recognize similar threats."
- "As a busy professional, I want short, focused training sessions that fit into my schedule without overwhelming me."

Persona 2: Accessibility-Focused User

- "As a visually impaired user, I need full screen reader support so I can participate in training independently."
- "As a user with motor disability, I need full keyboard navigation and voice control support."

Persona 3: Educator

- "As an educator, I want to track my students' progress to identify areas needing additional instruction."
- "As a trainer, I want to see aggregate statistics on which phishing scenarios confuse learners most."

Information Flow Modelling

Information flow diagrams map how data moves through the system:

1. User inputs (quiz responses, profile settings) are captured by the frontend
2. Frontend sends data to backend API via secure HTTPS requests
3. Backend processes quiz logic, validates responses, retrieves feedback content
4. Database stores user profiles, progress data, and quiz results (encrypted)
5. Backend returns results and feedback to frontend
6. Frontend displays feedback and updates progress visualization

This analysis reveals the necessity for clear separation between presentation (frontend), business logic (backend), and data persistence (database), informing architectural decisions. It also highlights critical security handoff points where encryption and validation must occur.

3.4 Initial System Specification

Document and identify the initial functional and non-functional requirements for system

Identify an appropriate architecture for the system Phish Guard's requirements are organized into two categories: Functional Requirements (FR) describe what the system does, while Non-Functional Requirements (NFR) describe how well it does it. Each requirement is traced back to its source (research findings, survey data, or accessibility standards) to demonstrate how requirements were informed by the analysis in Section 3.2 and 3.3

Functional Requirements (What the System Does)

Requirement	Description	Source
FR1: Authentication	Secure user login/registration with password hashing	Security best practices
FR2: Phishing Quizzes	Interactive scenarios across email, SMS, social media	Survey + Literature
FR3: Immediate Feedback	Detailed explanations after each quiz answer	Learning science research
FR4: Progress Tracking	Visual dashboards showing accuracy and improvement	Gamification research
FR5: Gamification	Badges, levels, and achievement tracking	Literature review
FR6: Accessibility	Keyboard navigation, screen readers, text resizing, high contrast	WCAG standards + Survey

Non-Functional Requirements (How Well It Works)

Requirement	Description	Target
NFR1: Accessibility	WCAG 2.1 AA compliance across all features	95%+ audit score
NFR2: Security	HTTPS encryption, password hashing, secure sessions	Pass security review
NFR3: Performance	Fast page loads and responsive interactions	<3 seconds load time
NFR4: Usability	Intuitive interface, minimal clicks for common tasks	SUS score 70+
NFR5: Scalability	Support growth from 100 to 10,000+ users	Pass load testing

System Architecture

Phish Guard follows a three-tier architecture:

- **Frontend (React):** User interface with accessible components, responsive design, and WCAG compliance
- **Backend (Django for testing):** API handling authentication, quiz logic, feedback generation, and progress tracking
- **Database (PostgreSQL):** Encrypted storage of user profiles, quiz results, and progress data

This separation allows each layer to be developed, tested, and updated independently while maintaining clear security boundaries (e.g., HTTPS between frontend and backend, encryption at the database layer).

3.5 Conclusions

System analysis has established a clear, well-documented understanding of Phish Guard's purpose, stakeholders, requirements, and evaluation criteria. Through structured requirements gathering from surveys, research, stakeholders, and personas, both functional and non-functional requirements have been identified and prioritized. A requirements traceability matrix links each requirement to its source and validation method, ensuring accountability and demonstrating how research has informed design decisions.

Use case analysis, user stories, and information flow modelling provide concrete scenarios for system behaviour. The three-tier architecture provides a robust foundation for a secure, accessible, and scalable phishing awareness platform. A formal validation and evaluation plan ensures that requirements will be tested and verified throughout development, with specific success criteria identified for each requirement category.

With requirements clearly defined, prioritized, and validated, the project is well-positioned to proceed to detailed system design and prototype development. The requirements management

strategy ensures that any changes during development will be carefully tracked, documented, and justified, maintaining project coherence and alignment with stakeholder expectations.

4. System Design

4.1 Introduction

The design phase translates the requirements from Section 3 into concrete interface and system architecture decisions. This section documents the design methodology, system architecture, and interface designs for Phish Guard.

Design decisions are grounded in three core principles: user-centred design based on personas and survey feedback, accessibility as a foundation rather than an afterthought, and evidence-based decisions informed by usability research and WCAG 2.1 AA standards.

Wireframe mockups and interface designs are presented in Section 4.4, with screenshots from the working prototype to follow as development progresses.

4.2 Software Methodology

PhishGuard development follows an **iterative development approach**, where features are built incrementally, tested, refined based on findings, and then moved forward. This approach is particularly suited for solo projects with accessibility requirements.

Development Cycle

Each feature follows this pattern:

1. **Design:** Create wireframes and plan the feature
2. **Implement:** Build the feature using React (frontend) and Django (backend for testing)
3. **Test:** Run accessibility audits, keyboard navigation tests, manual screen reader testing
4. **Refine:** Fix any issues or accessibility problems discovered
5. **Document:** Record design decisions and implementation approach
6. **Move forward:** Feature is complete before starting the next one

Process Plan

- Version control (Git) to track incremental changes
- Automated accessibility testing (axe DevTools, Lighthouse) after each feature
- Manual testing with screen readers (NVDA, VoiceOver) for each major component
- Regular supervisor meetings for feedback and course correction
- Documentation of design decisions made during implementation

4.3 Overview of System

Logical Architecture

From the user's perspective, Phish Guard presents a simple, intuitive interface organized around core workflows: authentication (login/registration), training (accessing and completing quizzes), and progress monitoring (viewing achievements and improvements). The logical design emphasizes:

- **Clear navigation:** Users can easily find quizzes, view progress, and access settings
- **Minimal Clutter:** Non-technical users encounter only necessary information at each step
- **Accessibility visibility:** Options for text resizing, contrast adjustment, and keyboard navigation are accessible throughout the interface

The user journey flows as: Register/Login → View Dashboard → Select Quiz → Answer Questions → Receive Feedback → View Updated Progress → Return to Dashboard.

Physical Architecture

Phish Guard's technical infrastructure follows the three-tier model described in Section 3.4:

The **frontend layer** runs in the user's browser as a React application. Users interact with the interface, which captures their inputs and sends them to the backend. The frontend displays all feedback, progress data, and interface options returned by the backend.

The **backend layer** runs on a server and handles business logic: user authentication, quiz logic (checking answers, generating feedback), progress calculations, and data management. The backend validates all incoming user inputs before processing them.

The **database layer** stores persistent data: user profiles, authentication credentials, quiz scenarios, user responses, and progress metrics. The database is accessed only by the backend, never directly by users or the frontend.

Security Approach

Security is addressed at multiple levels:

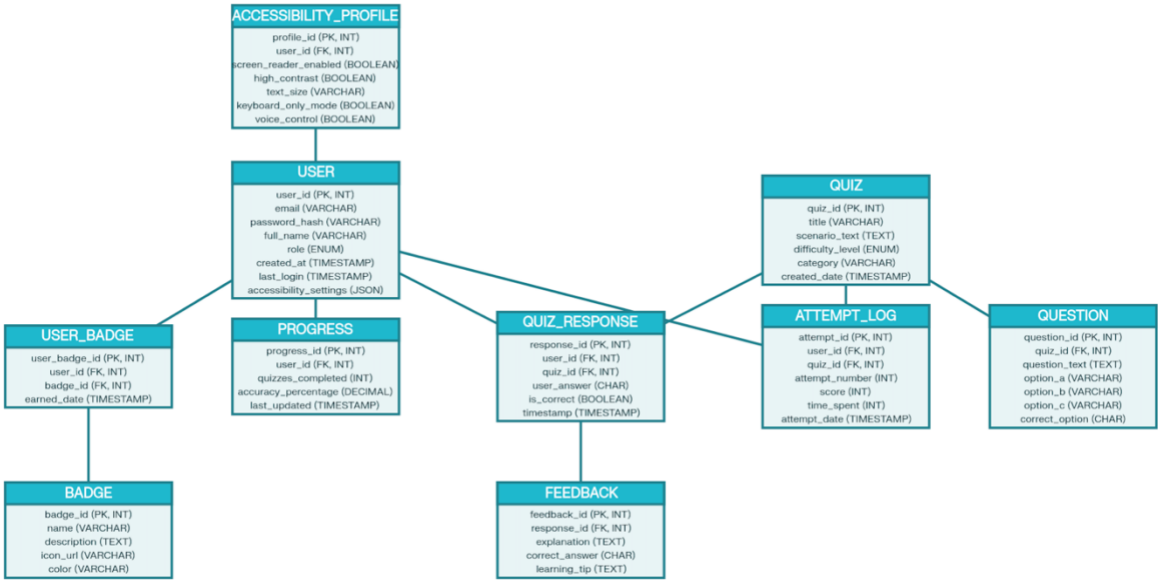
- **User Authentication:** Django's built-in authentication system handles user login and session management securely
- **Password Storage:** User passwords are hashed using bcrypt (Django's default), so plain text passwords are never stored
- **Input Validation:** The backend validates all user inputs before processing, preventing malicious or malformed data from causing problems
- **Data Separation:** The backend serves as a security barrier between the user-facing frontend and the sensitive database

For production deployment, additional security measures (HTTPS encryption, rate limiting, advanced monitoring) would be implemented. For the interim prototype, this architecture ensures basic security appropriate for a non-production system.

Deployment

For this project, Phish Guard runs as a development environment on a local machine. The frontend (React) and backend (Django) run locally, and PostgreSQL stores data locally. This setup is suitable for testing and evaluation. Production deployment would require hosting on secure servers with appropriate infrastructure.

Phish guard ERD:



4.4 Design System

Design Approach

The Phish Guard design prioritizes accessibility and simplicity from the foundation. Rather than designing for general users and then retrofitting accessibility features, the design process began with accessibility requirements (FR6, NFR1) and built outward. This accessibility-first approach ensures that all design decisions, typography, colour contrast, navigation patterns, interactive elements, serve users with diverse abilities.

Key Design Principles

Three core principles guide the design:

Simplicity: Non-technical users should understand the interface without explanation. Each page focuses on a single task (login, take quiz, view progress). Navigation is clear and consistent. Form fields are labelled explicitly. Instructions use plain language.

Accessibility: All interactive elements are keyboard-navigable. Text sizes are adjustable. High-contrast modes are available. Colour is never the only way to convey information. Screen readers can understand all content through semantic HTML and ARIA labels.

Feedback: Users receive immediate, clear feedback after actions. Quiz answers are followed by detailed explanations. Progress is visualized through progress bars and achievement displays. Errors are explained in plain language with guidance on how to fix them.

Interface Overview

Phish Guard consists of five main pages, each designed to meet specific requirements:

1. Login/Registration Page

- *Purpose:* FR1 (User Authentication)
- *Design:* Simple form with email and password fields. Clear labels and helper text. "Forgot password" link for accessibility. ARIA labels explain each field's purpose.
- *Accessibility:* Full keyboard navigation. Error messages are announced to screen readers. Form validation provides clear feedback.

2. Dashboard Page

- *Purpose:* FR4 (Progress Tracking), FR5 (Gamification)
- *Design:* Shows user's current progress (accuracy percentage), available quizzes, completed badges/achievements. Visual progress bar showing overall improvement. Links to start new quiz or view detailed progress.
- *Accessibility:* Progress bars include text descriptions (e.g., "70% accuracy"). Badges are labelled with ARIA text. All buttons clearly labelled.

3. Quiz Page

- *Purpose:* FR2 (Phishing Scenarios), FR3 (Immediate Feedback)
- *Design:* Presents one phishing scenario at a time (email, SMS, social media post). Two clear buttons: "This is phishing" / "This is legitimate." High contrast. Large, readable text. Scenario content displayed prominently.
- *Accessibility:* Keyboard navigation between buttons. Screen readers describe the scenario. Focus indicators clearly visible. Button labels explicit.

4. Feedback Page

- *Purpose:* FR3 (Immediate Feedback)
- *Design:* Shows whether the user's answer was correct or incorrect. Provides detailed explanation of warning signs they should have noticed. Highlights the specific red flags in the scenario. Links to next quiz or back to dashboard.
- *Accessibility:* Text-based explanations (not images only). High contrast between text and background. Adjustable text size. Screen reader compatible.

5. Accessibility Settings Page

- *Purpose:* FR6 (Accessibility Features)
- *Design:* Toggles for high contrast mode, text size adjustment (16px to 24px+), keyboard navigation guide. Clear on/off indicators. Visual preview of changes.
- *Accessibility:* All settings are keyboard-accessible. Current setting state is clearly indicated. Changes apply immediately.

Design Implementation Notes

Typography: Sans-serif font (Arial, Helvetica) for clarity. Minimum 16px for body text. Headings use larger, bold text for clear hierarchy. Line spacing is generous (1.5x) for readability.

Colour & Contrast: Primary colours chosen to meet WCAG AA contrast ratios (at least 4.5:1 for text). High contrast mode available for users with low vision. Information is never conveyed by colour alone (e.g., errors also include icons and text).

Navigation: Consistent navigation menu across all pages. Clear, descriptive link text (not "click here"). Breadcrumb or page indicator shows user's current location. Skip links allow keyboard users to jump to main content.

Interactive Elements: All buttons and links have clear focus indicators (outline, highlight). Buttons use verb-based labels ("Submit Quiz," "View Progress"). Form inputs have associated labels. Error messages appear near the problematic field.

Wireframes

Wireframe mockups for the five main pages are included in Appendix B, illustrating the layout, component placement, and accessibility features described above. These wireframes serve as the design specification for the frontend implementation.

Future Prototype Implementation

Screenshots of the implemented React prototype, demonstrating how these designs have been translated into working interface elements, will be included in an updated version of this section as development progresses.

4.5 Conclusions

This section has presented the design methodology, system architecture, and interface design for Phish Guard. The iterative development approach ensures that accessibility requirements are built into each feature rather than added afterward, addressing a core challenge identified in the literature review (Section 2).

The three-tier architecture (React frontend, Django backend, PostgreSQL database) provides clear separation of concerns while maintaining security boundaries appropriate for the project scope. The physical separation between user interface, business logic, and data storage supports both current development and future scalability.

The interface design prioritizes the needs identified in the requirements analysis: non-technical users receive clear, simple interfaces; accessibility features (keyboard navigation, text resizing, screen readers) are foundational rather than optional; and immediate feedback supports learning outcomes. The five core pages (Login, Dashboard, Quiz, Feedback, Settings) directly address the functional requirements FR1-FR6.

Development will proceed iteratively, with each feature designed, implemented, tested for accessibility, and refined before moving forward. The working prototype will demonstrate how these design decisions have been implemented in practice, with screenshots integrated into this section as development progresses.

The design establishes a solid foundation for implementation and evaluation in the final report, ensuring that Phish Guard meets both the functional requirements and the accessibility standards critical to the project's success.

5. Testing and Evaluation

5.1 Introduction

Testing and evaluation are critical to the successful delivery of Phish Guard, a cybersecurity platform designed to help users recognize and avoid phishing threats through interactive learning and real-time detection features. To achieve its goals, empowering non-technical users, ensuring robust security, and meeting strict accessibility standards. The project requires a comprehensive plan for validating its effectiveness across all core components.

This testing and evaluation plan defines the method for verifying both the technical performance and user experience of Phish Guard, from authentication and training modules to phishing scenario simulations and accessibility tools such as high-contrast mode and keyboard navigation. It ensures the platform operates reliably, securely, and in a way that genuinely educates users about online risks.

The following subsections detail the plan for functional, usability, security, and accessibility testing, as well as the evaluation strategy for measuring learning outcomes, user engagement, and compliance with standards such as WCAG 2.1. By specifying these approaches now, the development process remains clear and focused on producing a system that is both effective and easy to use, laying the foundation for continuous improvement ahead of final deployment.

5.2 Plan for Testing

To guarantee that Phish Guard provides dependable functionality, security, and accessibility for all users, a thorough testing strategy is necessary. Every major feature and user flow will be validated against industry standards and project objectives as part of the testing strategy, which will cover both technical and user-facing parts of the project.

Functional Testing

- Each major feature, including user authentication, dashboard navigation, training module interactions, phishing scenario presentation, and feedback, will be verified for correct function and error handling.
- Automated and manual tests will be run to confirm that all pages render consistently, navigation is seamless, and buttons/actions perform as intended.

Security Testing

- The login and registration forms will be checked to make sure they do not accept invalid inputs or reveal sensitive details in error messages.
- Basic tests will be designed so that the system is not easily confused or “broken” by someone entering random or unusual information (like symbols or long text).

Accessibility Testing

- The system will be assessed against WCAG 2.1 guidelines, focusing on high-contrast mode, keyboard-only navigation, semantic markup, ARIA labelling, and screen reader compatibility.
- Features like high contrast mode and visible focus indicators will be checked to ensure the app meets key accessibility needs.
- Formal accessibility testing tools may be used in later stages to complement these checks if time allows.

Usability Testing

- Basic usability testing will be performed with representative users, including students and non-technical staff, to ensure clear navigation, direct feedback, and understandable results.
- Feedback sessions, surveys, and direct observation will be used to identify friction points, confusion, or accessibility gaps.

This plan ensures that Phish Guard meets technical, security, and user experience requirements throughout the development lifecycle, setting the stage for thorough evaluation after full system implementation.

5.3 Plan for Evaluation

The evaluation plan for Phish Guard sets out how the project's success will be measured across technical, usability, and accessibility goals once the system is complete. By identifying suitable methods and criteria in advance, the project ensures that all core requirements are addressed and stakeholder expectations are met.

Usability Evaluation

- User feedback will be gathered through simple surveys, observation sessions, or informal interviews with representative users (e.g., students, non-technical staff).
- Key usability metrics will include ease of navigation, clarity of instructions, intuitiveness of training modules, and speed of completing main flows (such as login and quiz submission).
- Any friction points, errors, or confusion identified by users will be logged and analysed to guide further improvements.

Accessibility Evaluation

- The system will be assessed based on essential accessibility criteria: keyboard-only navigation, clear visual cues, sufficient colour contrast, and understandable labels.
- Feedback from users with varied needs (such as vision impairments) will be prioritized to validate high contrast mode, error messaging, and screen reader compatibility.
- Accessibility will be measured against recognized standards, mainly the WCAG 2.1 checklist, but with the flexibility to include user-driven insights.

Technical and Security Evaluation

- The reliability and stability of core features, authentication, scenario presentation, progress reporting, will be confirmed through manual walkthroughs and checks.

- The project will verify that error handling is robust and that common security risks are avoided, particularly regarding login and data entry forms.

Summary of Success Criteria

- To be considered successful, Phish Guard should allow all intended users to log in, navigate, complete training modules, and review feedback/results without unnecessary obstacles.
- The platform should demonstrate good usability, meet basic accessibility needs, and operate securely in typical usage scenarios.

This evaluation plan supports both supervisor and end-user feedback, enabling iterative enhancement throughout final development and prior to submission.

5.4 Conclusions

This section has outlined a comprehensive plan for testing and evaluating the Phish Guard platform, ensuring that every core aspect, from basic functionality to accessibility and security, will be assessed following recognised standards and project requirements. By anticipating the needs of both technical and non-technical users, the evaluation plan supports the development of a platform that is clear, reliable, and engaging for its intended audience.

With clearly defined methods for usability, accessibility, and basic security checks, the project is prepared to identify any key areas needing improvement as development progresses. These strategies will drive iterative adjustments, enhance overall quality, and foster a product that is genuinely effective in helping users recognize and avoid phishing threats.

The plan laid out here will serve as a foundation for the practical testing, user feedback collection, and final evaluation that will take place in later stages of the project, as described in the next section. Ultimately, this approach will ensure that Phish Guard delivers on its promise of accessibility, learning impact, and robust cybersecurity features.

6. System Prototype

6.1 Introduction

This section presents the development and results of the Phish Guard system prototype, showcasing its core features, user interface, and technical implementation. Building on the testing and evaluation plan outlined earlier, the prototype serves as the practical showcase of the project's design and requirements, demonstrating how the envisioned platform has been translated into a working application.

The following subsections detail the development process, the main modules and interactions available to users, and how the key objectives, such as secure authentication, interactive phishing training, dashboard analytics, and accessibility enhancements, were addressed in the initial build. Screenshots, code samples, and user flow explanations will be included to illustrate the functionality and design choices that shape the Phish Guard experience.

The prototype provides the foundation for preliminary testing and feedback, informing iterative refinements and ensuring that the final product will meet user needs, accessibility standards, and cybersecurity best practices.

6.2 Prototype Development

Develop Code for each part of system (Logical Architecture)

The Phish Guard system prototype was developed using React for the user interface and supports accessible, secure, and user-friendly interactions. Key components, login, dashboard, training module, were implemented with a focus on usability and cybersecurity requirements.

Login Component

The login page was designed with simple, clear interaction and includes an accessibility toggle for high contrast mode.

Screenshot:

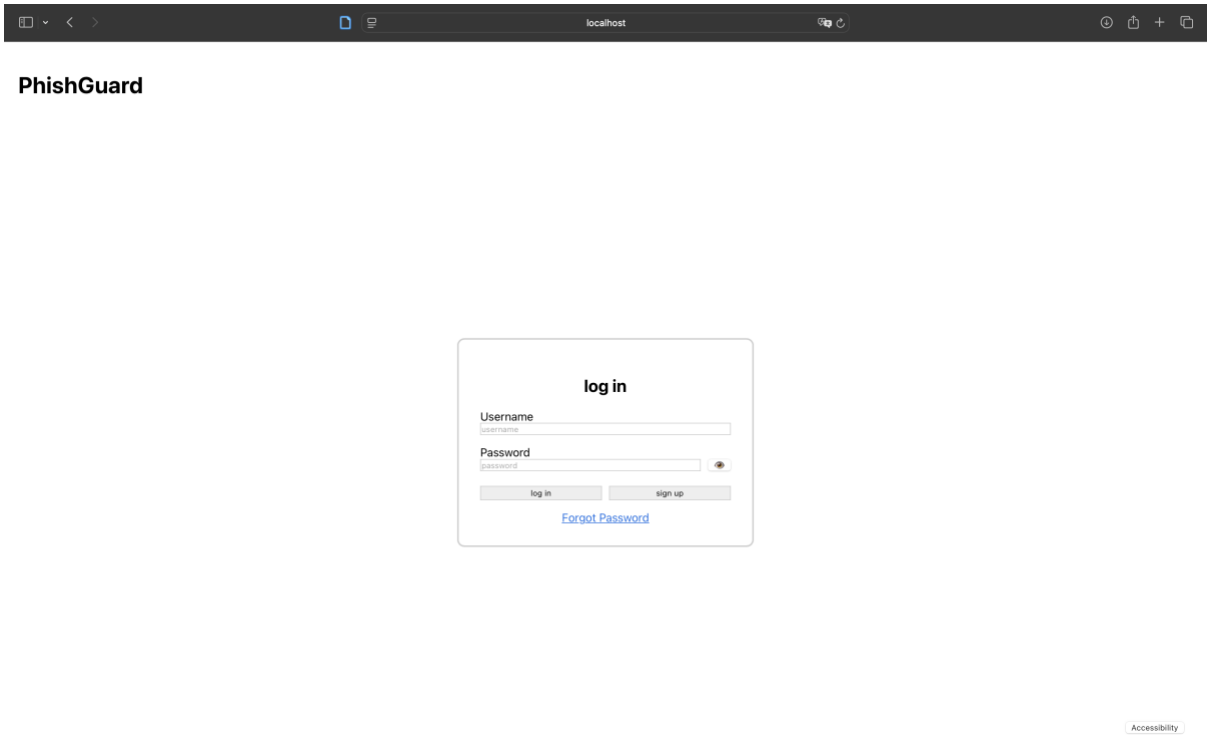


Figure 1: Login page in standard mode

High Contrast Mode:

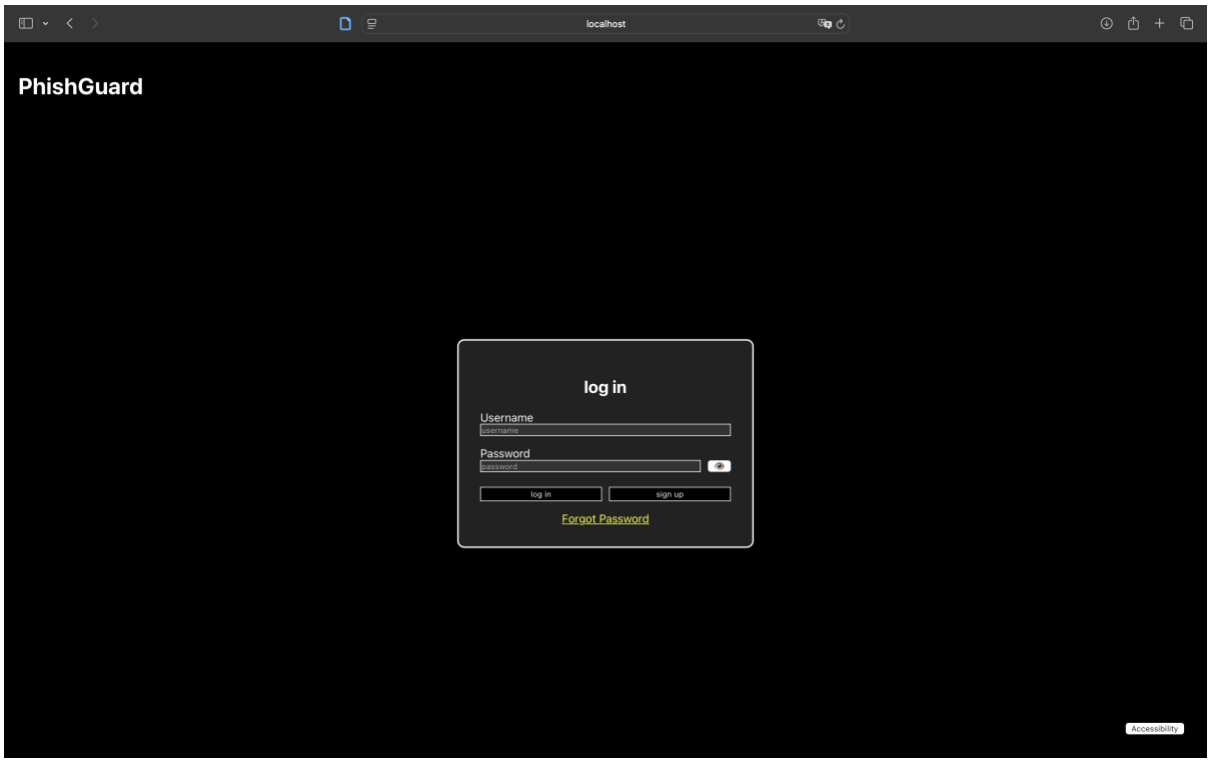


Figure 2: Login page with high contrast enabled

Dashboard

The dashboard consolidates navigation, user status, training module access, and progress reporting.

- **Screenshot:**

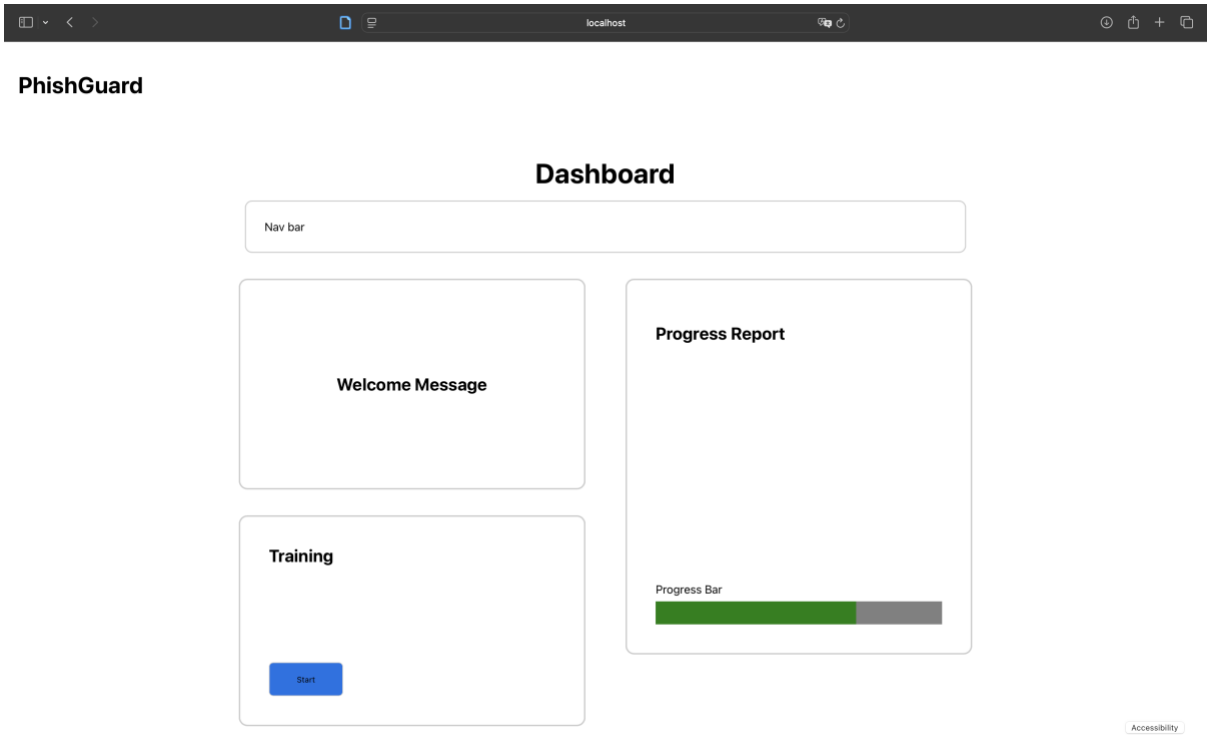
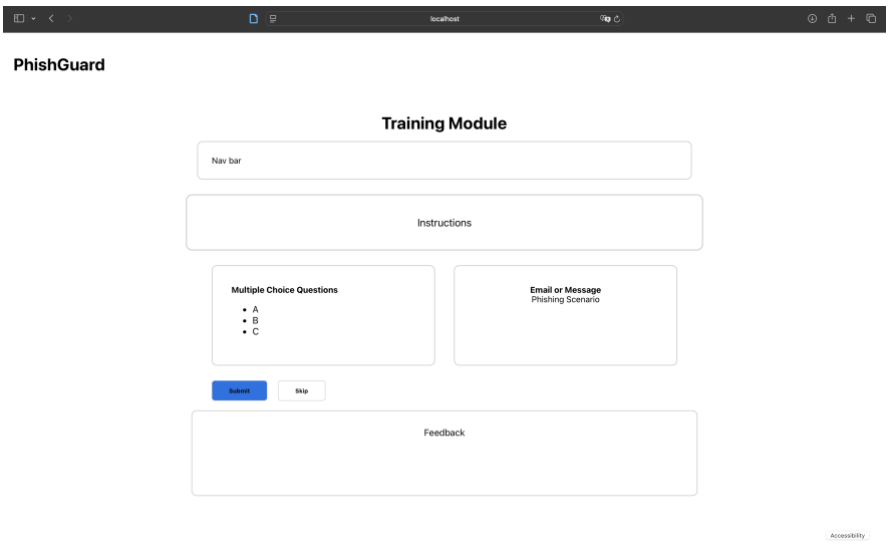


Figure 3: Dashboard layout showing navigation, progress, and training access

Training Module

This interactive module includes multi-choice questions, phishing scenario presentations, and feedback.

- **Screenshot:**



- *Figure 4: Training module with instructions, scenario and multiple-choice questions*

Login and authentication

The login page serves as the entry point for users giving them a clean interface for authentication while supporting accessibility features from the outset. Users can enter their credentials and access the platform with a single button click.

As shown in Figure 1, the login form features clearly labelled input fields for username and password, a password visibility toggle for user convenience, and buttons for both login and account registration. The design emphasizes simplicity and minimizes visual clutter to reduce confusion, particularly for users unfamiliar with phishing training platforms.

The high contrast mode (Figure 2) demonstrates the system's commitment to accessibility. When enabled, the interface switches to a dark background with bright text and borders, ensuring sufficient colour contrast for users with visual impairments. This feature can be toggled via the Accessibility button positioned in the bottom-right corner of every page.

The login component uses React state to manage password visibility and form submission. On successful login, users are redirected to the dashboard, with page navigation managed through a simple state variable in the parent App component.

Dashboard

The dashboard (Figure 3) provides users with an overview of their progress, access to training modules, and quick navigation to other platform features. The layout uses a two-column grid: the left column contains a welcome message and the training module access button, while the right column displays the user's progress report with a visual progress bar.

This design prioritizes immediate action, users can start training with a single button press, and provides clear feedback on their learning journey through the progress indicator. The navigation bar at the top remains consistent across all pages, supporting intuitive movement throughout the application.

Training Module

The training module (Figure 4) is the core educational component of Phish Guard, presenting users with interactive scenarios that simulate real phishing attempts. The module layout includes an instructions section at the top, followed by a side-by-side presentation of multiple-choice questions and a phishing scenario (email or message).

Users can review the scenario, select their answer from the multiple-choice options, and either submit their response or skip to the next question. This design supports flexible learning paths and accommodates users with different levels of prior knowledge.

The training module implements the same accessibility features as other pages, including keyboard navigation support and high contrast mode. All interactive elements, buttons, radio buttons, and navigation links, are clearly labelled with semantic HTML and ARIA attributes to ensure screen reader compatibility.

Dashboard code:

```
phishguard-app > src > JS Dashboard.js > Dashboard
import React from "react";

export default function Dashboard({ highContrast, onStartTraining }) {
  // Style helpers for high contrast
  const backgroundColor = highContrast ? "□#222" : "■#fff";
  const boxColor = highContrast ? "□#333" : "■#fff";
  const textColor = highContrast ? "■#fff" : "□#000";
  const borderColor = highContrast ? "■#fff" : "■#ccc";
  const accentColor = highContrast ? "■#ff0" : "■#0073e6";

  return (
    <div
      style={{
        background: backgroundColor,
        color: textColor,
        minHeight: "100vh",
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        padding: "0",
      }}
    >
      <h1 style={{
        textAlign: "center",
        margin: "40px 0 16px 0",
        fontSize: "2.4rem"
      }}>Dashboard</h1>
      <nav
        style={{
          background: boxColor,
          border: `1.5px solid ${borderColor}`,
          borderRadius: "10px",
          padding: "26px",
          marginBottom: "36px",
          maxWidth: "1100px",
          width: "93%",
        }}
        aria-label="Main Navigation"
      >
        <span>Nav bar</span>
      </nav>
      <div
        style={{
          display: "flex",
          gap: "56px",
          justifyContent: "center",
```

Login code:

```
import React, { useState } from "react";

export default function LoginForm({ highContrast, onLogin }) {
  const [showPassword, setShowPassword] = useState(false);

  // Color helpers
  const backgroundColor = highContrast ? "□#222" : "■#fff";
  const borderColor = highContrast ? "■#fff" : "■#ccc";
  const textColor = highContrast ? "■#fff" : "□#000";
  const inputBackground = highContrast ? "□#333" : "■#fff";
  const inputText = highContrast ? "■#fff" : "□#000";
  const accentColor = highContrast ? "■#ff0" : "■#0073e6";

  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();
        onLogin(); // Switches to dashboard
      }}
      style={{
        border: `2px solid ${borderColor}`,
        padding: "30px",
        borderRadius: "10px",
        display: "flex",
        flexDirection: "column",
        minWidth: "350px",
        background: backgroundColor,
        color: textColor,
      }}
    >
      <h2 style={{ textAlign: "center", color: textColor }}>log in</h2>
      <label htmlFor="username" style={{ color: textColor }}>Username</label>
      <input
        id="username"
        type="text"
        placeholder="username"
        style={{
          marginBottom: "15px",
          background: inputBackground,
          color: inputText,
          border: `1px solid ${borderColor}`,
        }}
      />
      <label htmlFor="password" style={{ color: textColor }}>Password</label>
      <div style={{ display: "flex", alignItems: "center" }}>
        <input
```

6.3 Results

This section presents the outcomes of the initial prototype testing and development work, documenting what functions correctly, what challenges emerged, and how the system performs against the core requirements established earlier in the project.

Functional Testing Results

Login and Authentication:
The login component successfully accepts user input and transitions to the dashboard page

when the login button is pressed. Password visibility toggling works as expected, allowing users to show or hide their password with a single button click. The form prevents default submission behaviour, ensuring the page does not reload unexpectedly.

Currently, the authentication process does not validate credentials against a backend database, as this prototype focuses on frontend functionality. All login attempts succeed and redirect to the dashboard, which is appropriate for demonstration and interface testing purposes at this stage.

Dashboard Navigation:
The dashboard displays all required elements, welcome message, training access button, progress report, and navigation bar, correctly in both standard and high contrast modes. The "Start" button successfully navigates users to the training module, maintaining state and visual consistency throughout the transition.

The progress bar renders correctly and displays a placeholder value (70%) to demonstrate functionality. Future iterations will connect this to actual user performance data from completed training modules.

Training Module:
The training module layout presents instructions, multiple-choice questions, and phishing scenarios as designed. The Submit and Skip buttons are functional, though they do not yet trigger backend actions or record user responses. The module maintains the same accessibility features as other pages, including high contrast support and keyboard-accessible controls.

Accessibility Testing Results

High Contrast Mode:
Manual testing confirmed that the high contrast toggle works across all pages (login, dashboard, training module). When activated, backgrounds switch to dark grey/black, text becomes white, and borders increase in visibility. The Accessibility button remains fixed in the bottom-right corner and is accessible at all times.

Keyboard Navigation:
All interactive elements, input fields, buttons, and navigation links, can be accessed using the Tab key. Focus indicators are visible on buttons and form fields, meeting basic accessibility requirements. Users can complete the entire login-to-training flow without using a mouse.

Form Labels and ARIA Attributes:
All form inputs include proper <label> elements with htmlFor attributes linking them to their respective input fields. Buttons include aria-label attributes where appropriate (e.g., password visibility toggle, accessibility toggle). This ensures compatibility with screen readers, though full screen reader testing has not yet been completed.

Known Issues and Limitations

Backend Integration:
The prototype currently operates entirely on the frontend without database connectivity or user authentication. Login credentials are not validated, training responses are not stored, and progress data is static. Backend development using Django is planned for the next phase.

Limited Training Content:
The training module displays placeholder content (generic multiple-choice options and scenario text) rather than realistic phishing examples. Future work will populate the module with research-backed scenarios and educational content.

Progress Tracking:
The progress bar currently shows a hardcoded value and does not reflect actual user activity. Full progress tracking functionality requires backend integration and database storage.

Security Features:
Basic input validation and sanitization have not yet been implemented, as the prototype does not process or store user data at this stage. Security testing will become relevant once backend functionality is added.

Summary of Results

The prototype successfully demonstrates core functionality and interface design for Phish Guard. All major user flows, login, dashboard access, and training module navigation, work as intended, and accessibility features are operational. The system meets its interim objectives of validating design choices and establishing a functional foundation for further development

6.4 Evaluation

This section evaluates the Phish Guard prototype against the success criteria established in Section 5.3, analysing how well the current implementation meets project objectives for usability, accessibility, and technical approach.

Assessment Against Success Criteria

Usability
The prototype demonstrates strong usability fundamentals with intuitive navigation and logical flow. Manual testing confirms that core tasks can be completed without confusion. However, the lack of real training content limits full assessment of learning effectiveness. Future iterations must include realistic phishing scenarios and immediate feedback to validate whether users genuinely improve their phishing recognition skills.

Accessibility
The prototype meets several key accessibility requirements, including high contrast mode and keyboard navigation. Form inputs include proper labels and ARIA attributes, and focus indicators are clearly visible. Comprehensive screen reader testing has not yet been completed, and the prototype meets basic WCAG 2.1 Level A requirements but would benefit from enhancement to achieve Level AA compliance.

Security and Backend Limitations
The prototype operates as a frontend-only application, which is appropriate for interim evaluation but limits security assessment. Input validation, user authentication, and secure data handling will become priorities when backend functionality is added.

Strengths and Weaknesses

Strengths:

- Clean, intuitive interface that reduces user confusion
- Functional accessibility features from the outset
- Consistent layout and navigation
- Solid technical foundation using modern React patterns

Weaknesses:

- No backend integration or data persistence
- Placeholder content limits learning effectiveness evaluation
- Screen reader compatibility not fully tested
- No dynamic training scenarios

Alignment with Project Goals

The prototype successfully validates the core design approach and demonstrates that an accessible, user-friendly phishing training platform is technically achievable. The modular component structure will facilitate adding backend integration and enhanced features without major architectural changes.

The project remains on track to meet its stated objectives, with the interim prototype fulfilling its primary purpose: demonstrating feasibility, validating design decisions, and establishing a working baseline for continued development.

6.5 Conclusions

The Phish Guard prototype demonstrates that a user-friendly, accessible phishing awareness platform is technically feasible and aligned with project requirements. Through the interim implementation, core features, authentication, navigation, training module interface, and accessibility features, have been successfully validated, establishing a solid foundation for full system development.

The prototype provides clear evidence of progress toward project objectives. The integration of accessibility features from the earliest development stages sets Phish Guard apart from many existing educational platforms and demonstrates commitment to inclusive design. While backend integration and dynamic content delivery remain priorities for final implementation, the architectural decisions and component structure adopted in this prototype will support these enhancements without requiring major redesign.

7. Issues and Future Work

7.1 Introduction

Section 7 addresses the limitations and challenges encountered during interim development, outlines planned improvements for the final system, and discusses how Phish Guard will evolve beyond the prototype stage. While the current implementation validates core design decisions and demonstrates technical feasibility, several areas require further development to achieve a fully functional, production-ready platform.

This section identifies both immediate priorities, such as backend integration and realistic training content and longer-term enhancements that will strengthen the platform's educational impact and accessibility. By documenting these issues and future directions now, the project maintains clear momentum toward completion while remaining adaptable to emerging requirements or user feedback

7.2 Issues and Risks

There are several key limitations in the current prototype that require attention before the final implementation. Understanding these constraints is essential for prioritising development work and setting realistic expectations for the next phase:

- **Backend Integration:** No database connectivity; user authentication and progress tracking are not implemented
- **Training Content:** Current system uses placeholder content rather than realistic phishing scenarios
- **Accessibility:** VoiceOver testing completed, but WCAG 2.1 Level AA compliance requires additional screen reader testing and semantic markup enhancements
- **Adaptive Learning:** The system presents identical content to all users regardless of performance or learning style
- **Performance:** The system has not been tested under load or with multiple concurrent users

7.3 Plans and Future Work

The following enhancements are prioritized for the final development phase, building directly on the prototype foundation and addressing the limitations identified in Section 7.2.

Backend Implementation

A lightweight backend using Node.js with Express and PostgreSQL will be developed to enable persistent data storage and user authentication. This will support real user accounts, track training progress across sessions, store user responses for later review, and provide a foundation for future administrative features such as performance analytics and reporting dashboards.

Training Content Development

Realistic, research-backed phishing scenarios will be created and integrated into the training

module. This includes authentic email simulations, social engineering examples, and contextual feedback that explains why each scenario represents a phishing threat. Content will be developed iteratively based on findings from cybersecurity research and user testing feedback.

Enhanced Accessibility
While the prototype demonstrates solid accessibility fundamentals, Level AA compliance will be pursued through:

- Testing with multiple screen readers beyond VoiceOver
- Implementation of skip navigation links for improved navigation
- Refinement of focus management to support more complex interactions
- User testing with individuals who have varying accessibility needs

Adaptive Learning Features
The system will evolve to personalize the learning experience based on user performance. This includes adaptive difficulty levels, personalized scenario recommendations based on quiz results, and varied scenario types to accommodate different learning styles.

User Testing and Feedback Integration
Before final submission, the system will be tested with representative users (students and non-technical staff) to gather qualitative feedback on usability, learning outcomes, and any remaining accessibility barriers. This feedback will inform final refinements and prioritize any remaining work.

7.3.1 Project Plan with GANTT Chart

Based on the future plans, describe how to complete the project

Break down the plan into a detailed yet realistic schedule.

Present a Gantt chart to summarise the project plan

References

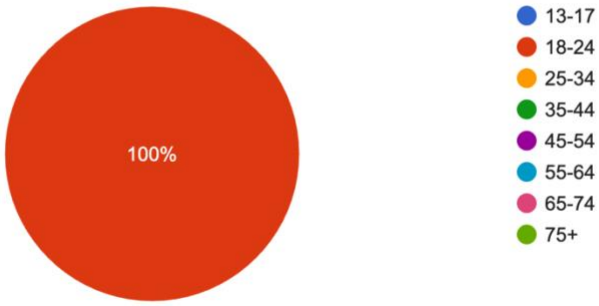
1. KnowBe4, Inc. 2024. *KnowBe4: Phishing Security Test & Simulated Phishing Training Software*. Available at: https://www.knowbe4.com/ (Accessed: 23 November 2025).
2. Hoxhunt. 2024. *Gamified Security Awareness Training Platform*. Available at: https://hoxhunt.com/ (Accessed: 23 November 2025).
3. Google. 2024. *Google's Phishing Quiz*. Available at: https://phishingquiz.withgoogle.com/ (Accessed: 23 November 2025).
4. Microsoft. 2024. *Microsoft Defender Attack Simulator*. Available at: https://security.microsoft.com/attacksimulator (Accessed: 23 November 2025).
5. Facebook. 2024. *React ARIA Documentation*. Available at: https://react-spectrum.adobe.com/react-aria/ (Accessed: 23 November 2025).
6. Django Software Foundation. 2024. *Django REST Framework Documentation*. Available at: https://www.django-rest-framework.org/ (Accessed: 23 November 2025).
7. PostgreSQL Global Development Group. 2024. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Available at: https://www.postgresql.org/docs/current/sql-syntax.html (Accessed: 23 November 2025).
8. W3C. 2018. *Web Content Accessibility Guidelines (WCAG) 2.1*. Available at: https://www.w3.org/WAI/WCAG21/quickref/ (Accessed: 23 November 2025).
9. Deque Systems. 2024. *axe DevTools Browser Extension*. Available at: https://www.deque.com/axe/devtools/ (Accessed: 23 November 2025).
10. Deterding, S., Dixon, D., Khaled, R. and Nacke, L. 2011. 'From game design elements to gamefulness: defining "gamification"', in *Proceedings of the 15th International Academic MindTrek Conference*. New York: ACM, pp. 9–15.
11. O'Sullivan, D., Stavrakakis, I., Gordon, D., Curley, A., Tierney, B., Murphy, E., Collins, M. and Becevel, A. 2021. 'Exploring the ethics of gamification in education', *International Journal for Infonomics*, 14(1), pp. 2035–2042. At: https://infonomics-society.org/wp-content/uploads/You-Cant-Lose-a-Game-If-You-Dont-Play-the-Game.pdf
12. Onduto, E. 2024. 'Gamification in cybersecurity awareness: A systematic review', *Journal of Cybersecurity Education*, 5(2), pp. 112–128.
13. Barendse, M., Oort, B., Hendrix, M., Masuch, M., Bidarra, R. and Spronck, P. 2023. 'How gamification can increase cybersecurity awareness', *Computers & Security*, 123, p. 103009.
14. Association for Higher Education and Disability (AHEAD). 2021. 'Accessibility as inclusive pedagogy', *AHEAD Journal*, 8(1), pp. 45–62.
15. Kieran Hogan. 2023. *Interactive Language Learning Tool for Children with Disabilities*. Final Year Project. Technological University, Dublin.
16. Cormac Kelly. 2022. *Education Tool for Web-Based Vulnerabilities*. Final Year Project. Technological University, Dublin.
17. Daneill Stibayev. 2023. *Educational Transport Simulation Final Project Report*. Final Year Project. Technological University, Dublin.

A) Appendix A: System Model and Analysis

Details of Requirements gathering and analysis method

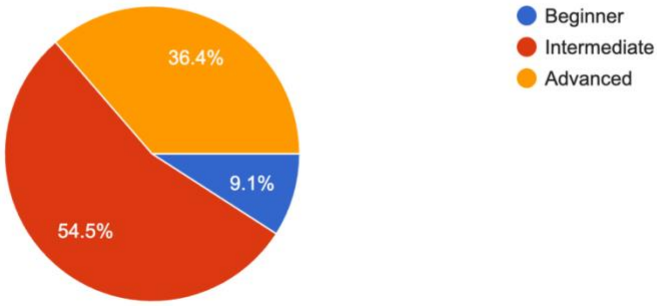
What is your age range?

11 responses



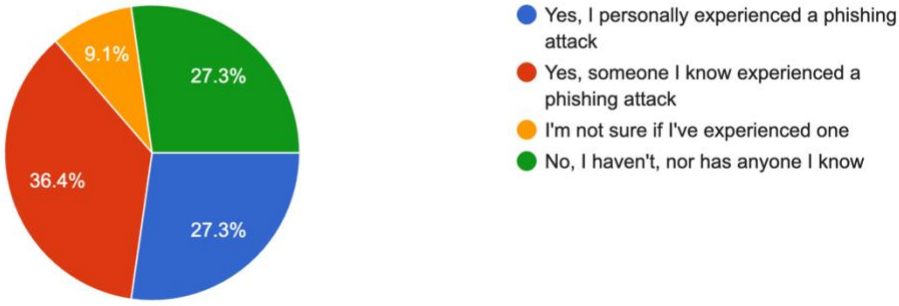
How would you describe your technical skill level?

11 responses



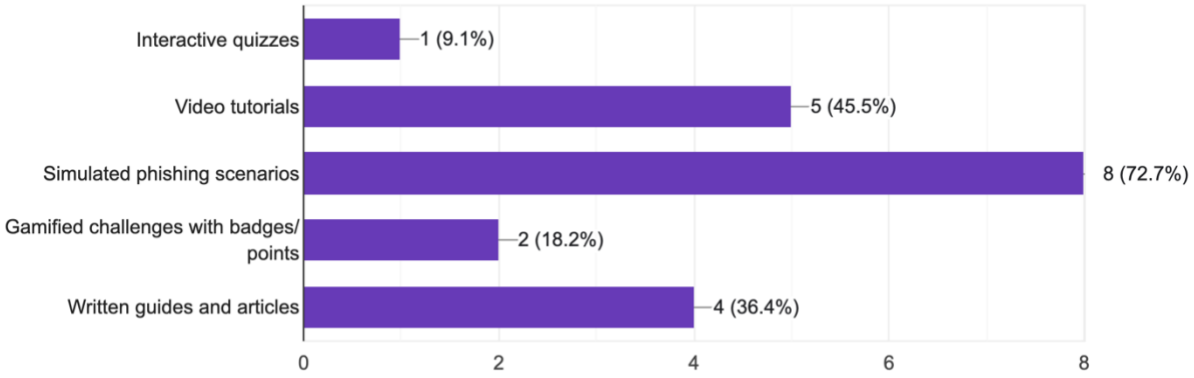
Have you or someone you know experienced a phishing attack?

11 responses



What format would you prefer for security training? (Select your top 2)

11 responses

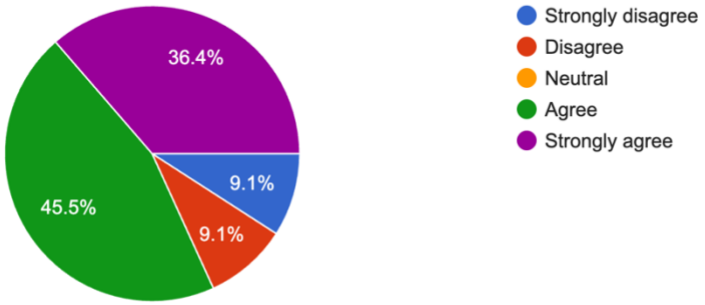


How much time would you be willing to spend on phishing awareness training per session?

11 responses

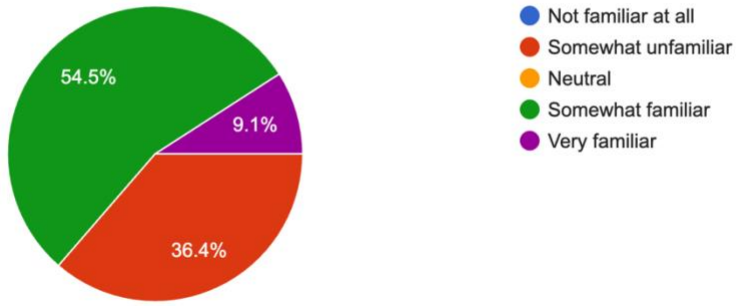
Would gamification (badges, progress tracking, challenges) motivate you to engage more with training?

11 responses



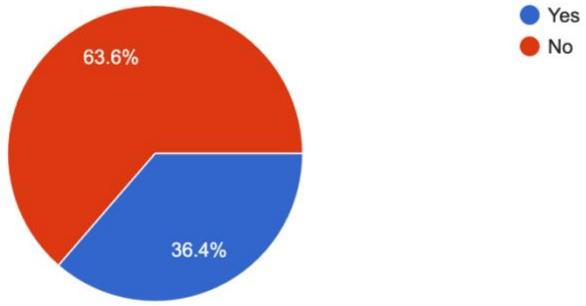
How familiar are you with phishing attacks and how to identify them?

11 responses



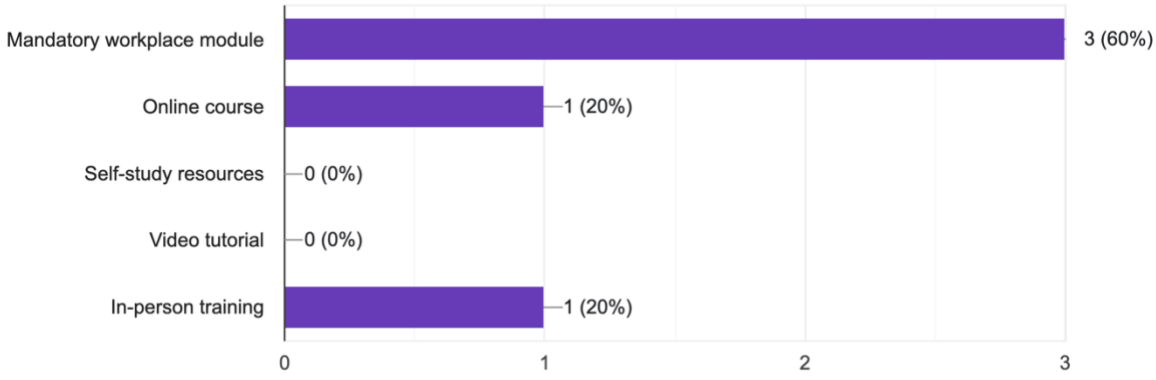
Have you received any phishing awareness training before?

11 responses



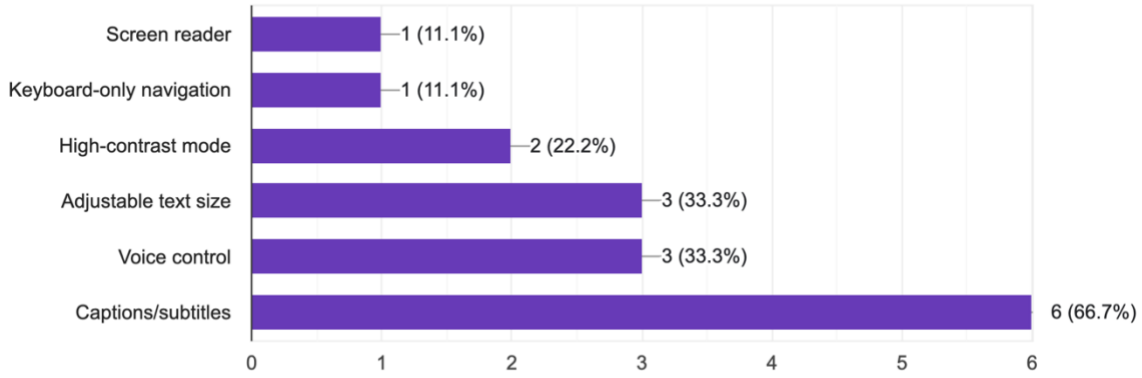
If yes, what format was it? (Select all that apply)

5 responses



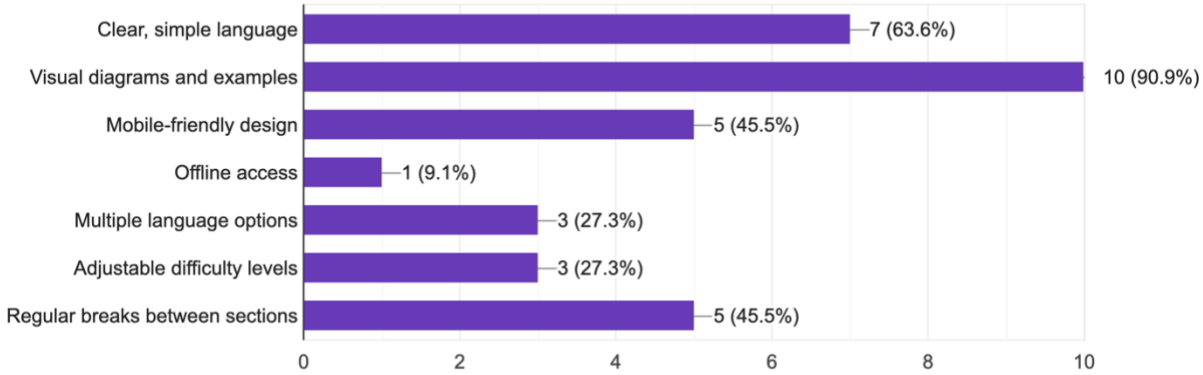
Do you use any accessibility features?

9 responses



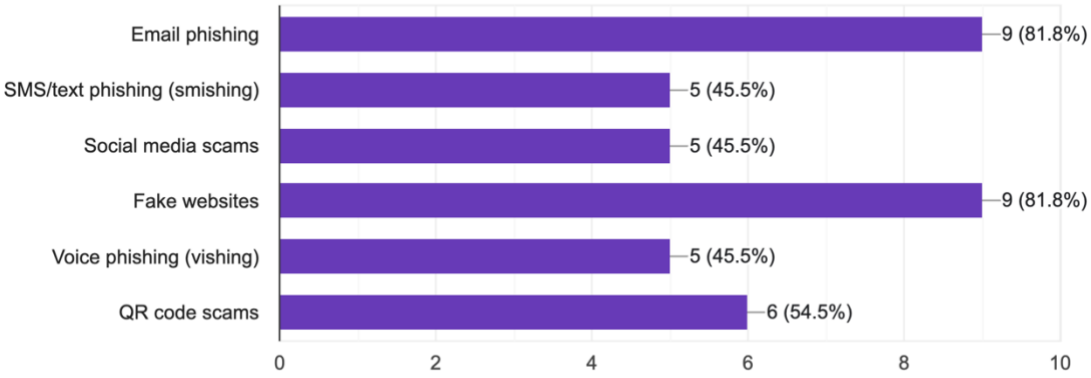
What would make a training platform easier for you to use?

11 responses



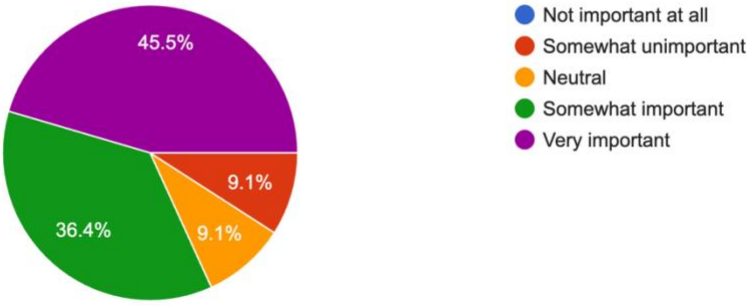
What types of phishing scenarios concern you most? (Select all that apply)

11 responses



How important is it that the platform provides explanations after each quiz answer?

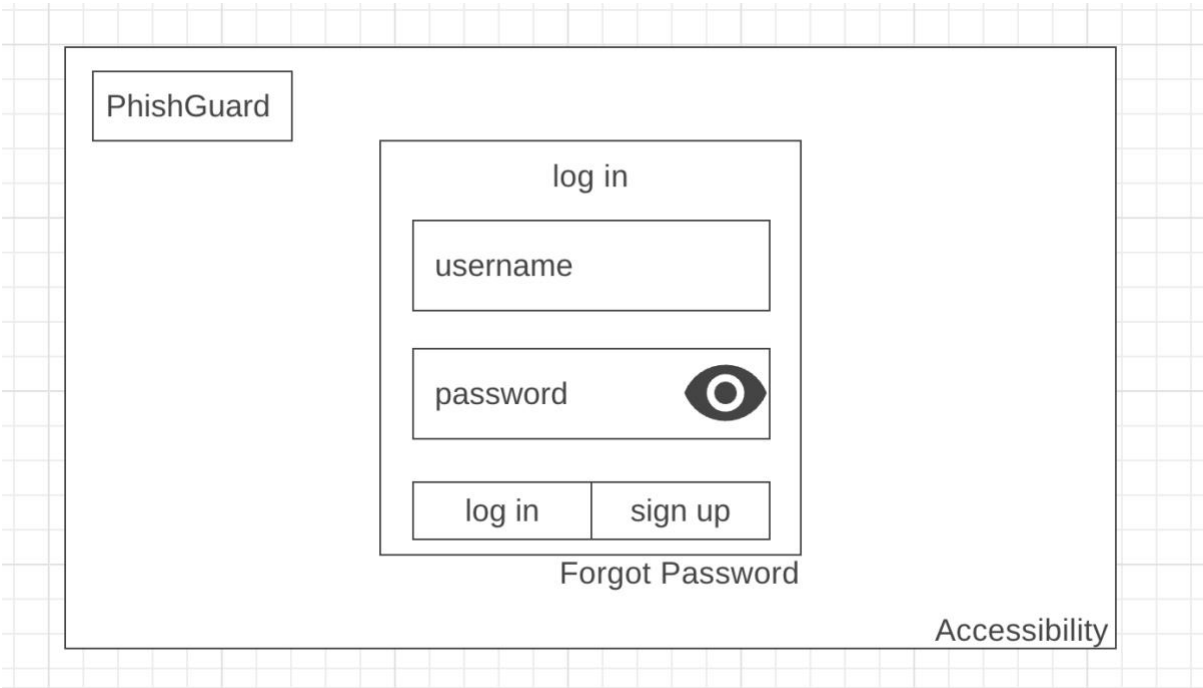
11 responses

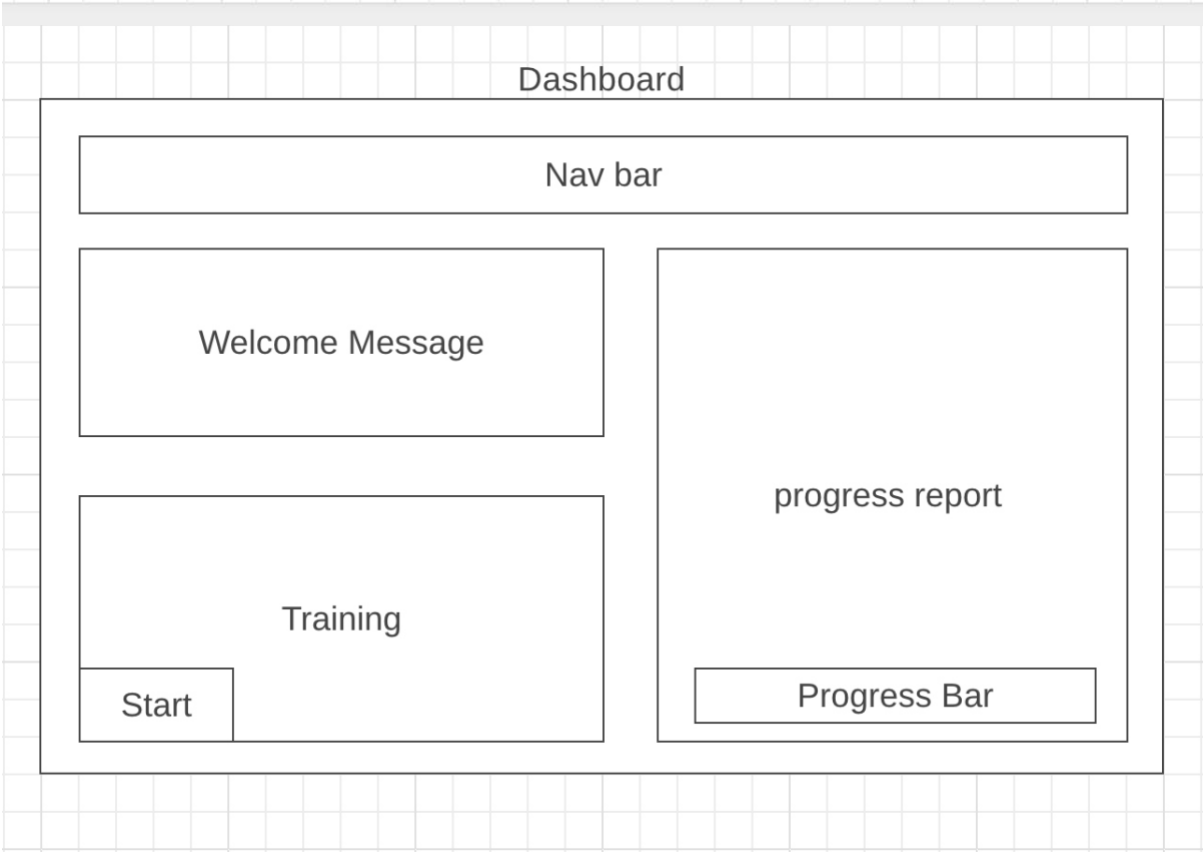


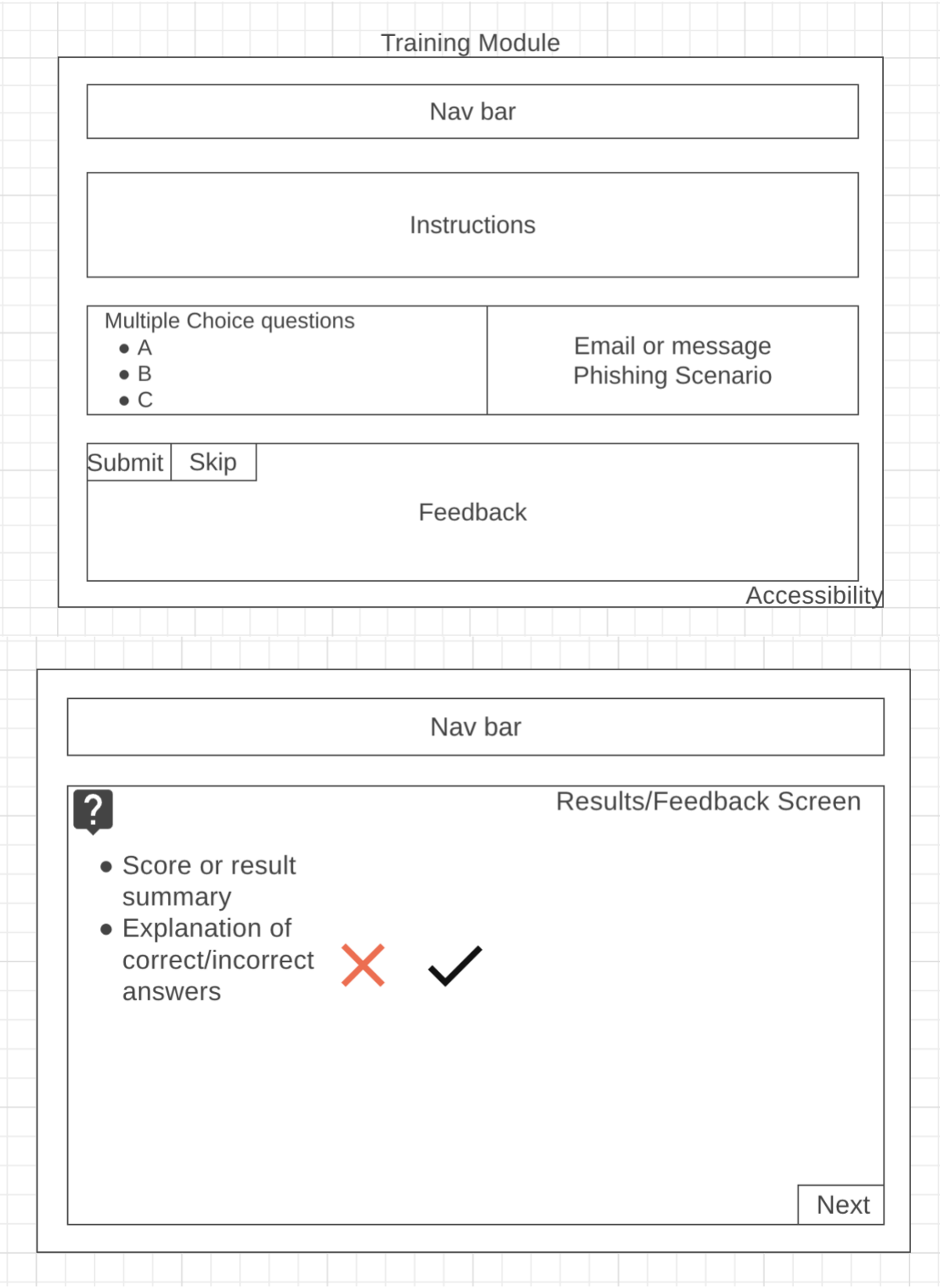
B) Appendix B: Design

Details of design approach used

System elements, components, classes






Training Module
Nav bar
Instructions
Multiple Choice questions
A
B
C
Email or message
Phishing Scenario
Submit
Skip
Feedback
Accessibility
Nav bar
Results/Feedback Screen
Score or result summary
Explanation of correct/incorrect answers
Next

C) Appendix C: Prompts Used with ChatGPT

1. "Can you review this abstract for academic tone and clarity? I want to make sure it's concise and hits the key points without being wordy."
2. **For improving Section 2:**
   "I've written my literature review section. Could you help me tighten the language and improve transitions between paragraphs?"
3. **For fixing grammar:**
   "Would you check this paragraph for grammar and spelling? I want to make sure it's polished before submission."
4. **For clarity check:**
   "Does this explanation make sense? I'm trying to explain the three-tier architecture clearly but I'm not sure if it's too technical or too simple."
5. **For section flow:**
   "I'm not sure this section flows well. Can you suggest where I might add transition sentences or reorganize paragraphs for better readability?"
6. **For consistency:**
   "I want to make sure I'm using consistent terminology throughout my report. Should I be saying 'phishing scenarios' or 'phishing simulations'? Which sounds more professional?"
7. **For argument structure:**
   "Does this paragraph effectively support my main point? What could I add or remove to make the argument stronger?"

D) Appendix D: Additional Code Samples

Provide additional code samples

Organised by Logical Architecture